

Gemini napíše celý program za pár sekund. Právě proto je snadné se nic nenaučit. Programovat se naučíš jen tím, že kód píšeš, spouštíš, děláš chyby a opravuješ je. AI použijvej jako trpělivého kolegu, který radí a vysvětluje – ne jako někoho, kdo píše za tebe.

Tři dobré role AI při programování

Role	Kdy	Vzorová věta
Vysvětlovač	nerozumíš cizímu kódu nebo příkladu z lekce	„Vysvětlí mi řádek po řádku, co dělá tento kód. Nic neměň.“
Detektiv chyb	program hlásí chybu nebo dělá něco jiného, než má	„Vysvětlí, co znamená tahle chybová hláška, a napověz, kde hledat. Opravu udělám já.“
Trenér	chceš si procvičit	„Dej mi úlohu na cykly na můj úrovni a pak zkontroluj moje řešení.“

Vzorové dotazy

Vysvětlí kód (STEM 1.+2. – Cykly a seznamy)

Učím se Python (cykly a seznamy). Nerozumím tomuto kódu:

```
teploty = [12, 15, 9, 14]
soucet = 0
for t in teploty:
    soucet = soucet + t
print(soucet / len(teploty))
```

Vysvětlí mi, co se děje v každém průchodu cyklem. Pak se mě zeptej, co program vypíše, a nech mě odpovědět.

Najdi chybu (STEM 1.+2. – Testování a ladění)

Můj program má spočítat obsah obdélníku, ale hlásí chybu:

```
a = input('Strana a: ')
b = input('Strana b: ')
print(a * b)
```

TypeError: can't multiply sequence by non-int of type 'str'

Neopravuj mi kód. Vysvětlí, co ta chybová hláška znamená, a polož mi otázku, která mě navede k opravě.

Nápověda místo řešení (STEM 1.+2. – Větvení: řešič rovnic)

Mám napsat program, který načte a, b, c a vypíše řešení kvadratické rovnice, i když rovnice nemá řešení. Nechci hotový kód. Rozepiš mi úlohu na 4–5 kroků slovy a u každého kroku mi dej jen nápovědu, jakou konstrukci Pythonu použít. Kód napíšu sama a pak ti ho pošlu ke kontrole.

Testy k hotovému programu (STEM 1.+2. – Testování a ladění)

Tady je můj hotový program: ... Nepřepisuj ho. Navrhni mi 5 testovacích vstupů včetně okrajových případů (nula, záporné číslo, prázdný seznam) a napiš, co by měl program pro každý vstup vypsát. Spustím to sám a porovnám.

Databáze (STEM 3.+4. – Dotazy nad databází)

Mám tabulky kniha(id, nazev, zanzr) a vypujcka(id, kniha_id, datum) s vymyšlenými daty. Chci dotaz SQL, který vypíše počet výpůjček v každém žánru. Můj pokus: SELECT zanzr, COUNT(*) FROM kniha GROUP BY zanzr; – jenže počítá knihy, ne výpůjčky. Nenapiš mi správný dotaz, jen mi napověz, co v něm chybí.

Python a Google Colab

- Kód spouštěj v Google Colabu se školním účtem – notebook se uloží na tvůj Disk a odkaz na něj můžeš dát do portfolia.
- Piš po malých kouscích a každý kousek hned spusť. Chybu v pěti řádcích najdeš snáz než v padesáti.
- Než se zeptáš AI, přečti si chybovou hlášku. Poslední řádek říká typ chyby (NameError, TypeError, IndentationError...), číslo řádku ukazuje, kde hledat.
- Průběh programu si můžeš krokovat v Python Tutoru – uvidíš, jak se mění hodnoty proměnných.
- Když ti editor nebo Colab nabídne, že kód dopíše AI, při učení ho nenech psát celé řešení. Vysvětlit chybu si ale nechat můžeš.

Pravidlo 3 minut

Když kód z AI nedokážeš do 3 minut vysvětlit řádek po řádku vlastními slovy, není tvůj a jako doklad do portfolia nepatří. Buď si ho nech vysvětlit a napiš ho znovu sám nebo sama, nebo ho v portfoliu přiznej jako kód z AI (návod 07).

Bezpečnost

- Do chatu nikdy nevkládej hesla, přístupové klíče (API klíče, tokeny) ani přihlašovací údaje – ani uvnitř kódu. Nahraď je textem HESLO_SEM.
- Kód z AI spouštěj, jen když rozumíš, co dělá. Pozor hlavně na kód, který maže soubory, stahuje něco z internetu, instaluje balíčky nebo mění nastavení počítače.
- Instaluj jen známé balíčky. AI si občas vymyslí název knihovny, která neexistuje – a pod podobným názvem může existovat podvodný balíček.
- Do chatu nevkládej skutečná data o lidech (jména, e-maily, výsledky). Na testy si vymysli fiktivní data.
- U webu a databází (STEM 3.+4.) se AI ptej i na bezpečnost: „Je v mém kódu riziko SQL injection?“

Než použiješ cizí kód

Našel jsem na internetu tuhle funkci pro kontrolu síly hesla: ... Než ji použiju, vysvětlí mi, co dělá, jestli něco neposílá po síti nebo neukládá do souboru a jestli je bezpečná. Nic nepřepisuj.

Licence a cizí kód

- Kód z AI může být podobný existujícímu kódu s licencí. Když Gemini cituje delší kód z nějakého repozitáře, může u zdroje uvést i licenci open source – zapiš si ji.
- Převzatý kód (z internetu i z AI) v programu označ komentářem, například: # podle nápovědy Gemini, upraveno.
- Licence open source (například MIT nebo GPL) použití obvykle povolují, ale vyžadují zachovat jméno autora a text licence; GPL navíc určuje, pod jakou licencí smíš šířit svůj program.
- Obrázky a texty vytvořené AI v projektu vždy označ. Nikdy nevydávej cizí kód ani výstup AI za svůj.

Zdroje a odkazy

- **Google Colab** – notebooky s Pythonem v prohlížeči, ukládají se na Google Disk
<https://colab.research.google.com/>
- **Nauč se Python – Print a chybové hlášky (PyLadies)** – jak číst chybovou hlášku Pythonu, než se zeptáš AI
<https://naucese.python.cz/course/pyladies/beginners/print/>
- **Python Tutor – krokování programu** – uvidíš, jak se při běhu programu mění proměnné
<https://pythontutor.com/visualize.html>
- **Nápověda Google – Zobrazení souvisejících zdrojů z aplikací s Gemini** – tlačítko Zdroje pod odpovědí, citace webu a kódu včetně licence open source
<https://support.google.com/gemini/answer/14143489?hl=cs>
- **Google blog – Gemini Guided Learning: How Google's new AI feature works** – jak režim vede žáka otázkami, u matematiky i u ladění kódu (anglicky)
<https://blog.google/products-and-platforms/products/gemini/guided-learning-google-gemini/>